Лабораторная работа №1

Задача 2. Параллельное решение задачи N-тел методом Эйлера (pthreads)

1. Постановка задачи

Дано N материальных точек с массами m_k, начальными координатами r_k и скоростями v_k, k = 1..N. Требуется определить траектории всех частиц на отрезке времени [0, t_end].

Вычисления распараллелены с помощью потоков pthreads: пары тел распределяются между потоками, силы считаются одновременно, затем результаты сводятся.

Формат ввода. Из командной строки программа получает число потоков, конечное время t_end, шаг dt и имя файла с начальными данными. Первая строка файла содержит число частиц n; далее n строк по 7 чисел: масса, 3 координаты, 3 компоненты скорости.

Формат вывода. CSV-файл; каждая строка соответствует одному моменту времени и содержит t x1 y1 z1 x2 y2 z2 ... xn yn zn.

2. Математическая модель

2.1. Закон всемирного тяготения

Сила, действующая на тело q со стороны тела k:

$$F_{qk} = \frac{G m_q m_k}{\left(r_k - r_q\right)^3} \cdot \left(r_k - r_q\right)$$

где $G = 6.67430 \cdot 10^{-11}$ м³/(кг·с²) - гравитационная постоянная. Суммарная сила на тело q получается суммированием по всем остальным телам k:

$$F_q = Gm_q \sum_{k \neq q} \frac{m_k}{(r_k - r_q)^3}(r_k - r_q)$$

Согласно второму закону Ньютона, m_q · d²r_q/dt² = F_q.

2.2. Метод Эйлера первого порядка

Уравнение второго порядка сводится к системе двух уравнений первого порядка:

$$\frac{dr_q}{dt} = v_q$$

$$m_q \frac{dv_q}{dt} = F_q$$

Замена производных конечными разностями даёт итерационные формулы (индекс n - номер шага, dt - шаг интегрирования):

$$x_q^n = x_q^{n-1} + v_{xq}^{n-1} dt$$

$$v_{xq}^n = v_{xq}^{n-1} + \frac{F_{xq}^{n-1}}{m_q} dt$$

Важно: в формуле (1) используется скорость со старого шага v^(n-1). Поэтому в программе сначала сохраняется старая скорость, по ней обновляется координата, и только затем обновляется скорость. Сила F^(n-1) вычисляется один раз в начале шага по текущим координатам.

2.3. Использование третьего закона Ньютона

Справедливо F_qk = -F_kq: сила, с которой тело q действует на тело k, равна по величине и противоположна по направлению силе, с которой k действует на q.

Поэтому сила для каждой пары тел (q, k) вычисляется один раз и применяется к обоим телам с противоположными знаками. Это вдвое сокращает число вычислений сил и является ключевым требованием задания.

3. Полный текст программы

Ниже приведён полный исходный код программы nbody.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <pthread.h>
#include <string.h>
#include "timer.h"

/* Гравитационная постоянная, м^3 / (кг * с^2) */
const double G = 6.67430e-11;

int n;                /* число частиц */
int thread_count;     /* число потоков */
double t_end;         /* конечный момент времени */
double dt;            /* шаг по времени */
int nsteps;           /* число шагов интегрирования */

double *m;            /* массы,                    длина n   */
double *pos;          /* координаты,               длина 3*n (x,y,z на частицу)
*/
double *vel;          /* скорости,                 длина 3*n */
double *force;        /* суммарная сила на частицу, длина 3*n */
double **lforce;      /* локальные силы по потокам, thread_count * 3*n */

int *par_start, *par_end;  /* блок частиц каждого потока (сведение и обновление)
*/
int *frc_start, *frc_end;  /* блок "строк" q каждого потока (этап сил) */

FILE *outfp;          /* выходной CSV-файл */

/* Многоразовый барьер на мьютексе + условной переменной.
 * Использует счётчик «поколений» (generation), чтобы поток, проснувшийся
 * поздно, не «прилип» к следующему поколению барьера (иначе - дедлок). */
pthread_mutex_t bmutex;
pthread_cond_t  bcond;
int bcount;
int bthreads;
int bgeneration;

void barrier_wait(void) {
    pthread_mutex_lock(&bmutex);
    int gen = bgeneration;
    bcount++;
    if (bcount == bthreads) {
        bgeneration++;
        bcount = 0;
        pthread_cond_broadcast(&bcond);
    } else {
        while (gen == bgeneration)
            pthread_cond_wait(&bcond, &bmutex);
    }
    pthread_mutex_unlock(&bmutex);
}

void Usage(char *prog) {
```

```
    fprintf(stderr,
        "usage: %s <nthreads> <t_end> <dt> <input_file> [output_file]\n", prog);
    exit(1);
}

/* Число пар, покрытых строками q = 0..q-1 (для балансировки этапа сил) */
static long cum_pairs(int q) {
    return (long) q * (n - 1) - (long) q * (q - 1) / 2;
}

/* Наименьшее q, при котором cum_pairs(q) >= target */
static int find_q(long target) {
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (cum_pairs(mid) >= target) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

/* Чтение входного файла:
 *   первая строка: n
 *   далее n строк: масса x y z vx vy vz */
int read_input(char *fname) {
    FILE *f = fopen(fname, "r");
    if (!f) { perror("input file"); return 0; }
    if (fscanf(f, "%d", &n) != 1) { fclose(f); return 0; }

    m     = malloc(n      * sizeof(double));
    pos   = malloc(3 * n  * sizeof(double));
    vel   = malloc(3 * n  * sizeof(double));
    force = malloc(3 * n  * sizeof(double));

    for (int q = 0; q < n; q++) {
        if (fscanf(f, "%lf %lf %lf %lf %lf %lf %lf",
                   &m[q], &pos[3*q], &pos[3*q+1], &pos[3*q+2],
                   &vel[3*q], &vel[3*q+1], &vel[3*q+2]) != 7) {
            fclose(f);
            return 0;
        }
    }
    fclose(f);
    return 1;
}

/* Вывод одной строки CSV: t x1 y1 z1 x2 y2 z2 ... xn yn zn */
void write_row(double time) {
    fprintf(outfp, "%.10g", time);
    for (int q = 0; q < n; q++)
        fprintf(outfp, " %.10g %.10g %.10g", pos[3*q], pos[3*q+1], pos[3*q+2]);
    fprintf(outfp, "\n");
}

/* Диагностика: суммарная энергия и суммарный импульс */
void diagnostics(const char *label) {
    double KE = 0.0, PE = 0.0;
    double P[3] = {0.0, 0.0, 0.0};
    for (int q = 0; q < n; q++) {
```

```
        double v2 = vel[3*q]*vel[3*q] + vel[3*q+1]*vel[3*q+1]
                  + vel[3*q+2]*vel[3*q+2];
        KE += 0.5 * m[q] * v2;
        P[0] += m[q] * vel[3*q];
        P[1] += m[q] * vel[3*q+1];
        P[2] += m[q] * vel[3*q+2];
        for (int k = q + 1; k < n; k++) {
            double dx = pos[3*k]   - pos[3*q];
            double dy = pos[3*k+1] - pos[3*q+1];
            double dz = pos[3*k+2] - pos[3*q+2];
            PE -= G * m[q] * m[k] / sqrt(dx*dx + dy*dy + dz*dz);
        }
    }
    printf("%s: total energy E = %.6e   momentum P = (%.3e, %.3e, %.3e)\n",
           label, KE + PE, P[0], P[1], P[2]);
}

void *thread_work(void *arg) {
    long t = (long) arg;
    double *lf = lforce[t];
    int q, k, dim, u;

    for (int step = 1; step <= nsteps; step++) {
        /* 1) обнуляем локальные силы данного потока */
        memset(lf, 0, 3 * n * sizeof(double));

        /* 2) этап сил: пары (q,k), q < k, q - из блока данного потока.
         *    Сила между q и k вычисляется один раз и применяется к обоим
         *    телам (закон Ньютона 3-го: F_qk = -F_kq). Это вдвое сокращает
         *    число вычислений. */
        for (q = frc_start[t]; q < frc_end[t]; q++) {
            double mq   = m[q];
            double px   = pos[3*q];
            double py   = pos[3*q+1];
            double pz   = pos[3*q+2];
            for (k = q + 1; k < n; k++) {
                double dx = pos[3*k]   - px;
                double dy = pos[3*k+1] - py;
                double dz = pos[3*k+2] - pz;
                double d2 = dx*dx + dy*dy + dz*dz;
                double inv3 = 1.0 / (d2 * sqrt(d2));   /* 1/r^3 */
                double f = G * mq * m[k] * inv3;
                double fx = f * dx, fy = f * dy, fz = f * dz;
                lf[3*q]   += fx;  lf[3*q+1]   += fy;  lf[3*q+2]   += fz;
                lf[3*k]   -= fx;  lf[3*k+1]   -= fy;  lf[3*k+2]   -= fz;
            }
        }

        barrier_wait();   /* все локальные силы готовы */

        /* 3) сведение локальных сил и интегрирование (метод Эйлера) */
        for (q = par_start[t]; q < par_end[t]; q++) {
            double mq = m[q];
            for (dim = 0; dim < 3; dim++) {
                double F = 0.0;
                for (u = 0; u < thread_count; u++)
                    F += lforce[u][3*q + dim];
                force[3*q + dim] = F;
                double old_v = vel[3*q + dim];
```

```
                pos[3*q + dim]  += old_v * dt;            /* x^n = x^(n-1) +
v^(n-1)*dt */
                vel[3*q + dim]  = old_v + (F / mq) * dt;  /* v^n = v^(n-1) +
(F/m)*dt */
            }
        }

        barrier_wait();   /* позиции обновлены */

        /* 4) вывод строки выполняет нулевой поток */
        if (t == 0) write_row(step * dt);
    }
    return NULL;
}

int main(int argc, char **argv) {
    if (argc < 5 || argc > 6) Usage(argv[0]);
    thread_count = atoi(argv[1]);
    t_end = atof(argv[2]);
    dt    = atof(argv[3]);
    char *infile  = argv[4];
    char *outfile = (argc == 6) ? argv[5] : "trajectories.csv";

    if (thread_count < 1 || dt <= 0.0 || t_end <= 0.0) Usage(argv[0]);
    if (!read_input(infile)) return 1;

    nsteps = (int) ceil(t_end / dt);
    if (nsteps < 1) nsteps = 1;

    /* блоки частиц для сведения/обновления */
    par_start = malloc(thread_count * sizeof(int));
    par_end   = malloc(thread_count * sizeof(int));
    for (int t = 0; t < thread_count; t++) {
        par_start[t] = (int)((long) t * n / thread_count);
        par_end[t]   = (int)((long)(t + 1) * n / thread_count);
    }

    /* блоки строк q для этапа сил (равное число пар на поток) */
    frc_start = malloc(thread_count * sizeof(int));
    frc_end   = malloc(thread_count * sizeof(int));
    long total_pairs = (long) n * (n - 1) / 2;
    for (int t = 0; t < thread_count; t++) {
        frc_start[t] = find_q(total_pairs * t / thread_count);
        frc_end[t]   = find_q(total_pairs * (t + 1) / thread_count);
    }

    lforce = malloc(thread_count * sizeof(double *));
    for (int t = 0; t < thread_count; t++)
        lforce[t] = calloc(3 * n, sizeof(double));

    outfp = fopen(outfile, "w");
    if (!outfp) { perror("output file"); return 1; }

    write_row(0.0);          /* начальное состояние (t = 0) */

    printf("n=%d threads=%d steps=%d dt=%.6g t_end=%.6g out=%s\n",
           n, thread_count, nsteps, dt, t_end, outfile);
    diagnostics("initial");
```

```
    pthread_mutex_init(&bmutex, NULL);
    pthread_cond_init(&bcond, NULL);
    bthreads = thread_count;
    bcount   = 0;
    bgeneration = 0;

    pthread_t *th = malloc(thread_count * sizeof(pthread_t));
    double start, finish;
    GET_TIME(start);
    for (long t = 0; t < thread_count; t++)
        pthread_create(&th[t], NULL, thread_work, (void *) t);
    for (int t = 0; t < thread_count; t++)
        pthread_join(th[t], NULL);
    GET_TIME(finish);

    diagnostics("final");
    printf("Elapsed time = %e s\n", finish - start);

    fclose(outfp);
    pthread_cond_destroy(&bcond);
    pthread_mutex_destroy(&bmutex);
    for (int t = 0; t < thread_count; t++) free(lforce[t]);
    free(lforce); free(frc_start); free(frc_end);
    free(par_start); free(par_end); free(th);
    free(force); free(vel); free(pos); free(m);
    return 0;
}
```

4. Пояснения к коду

4.1. Общая структура

Программа состоит из следующих частей: чтение входных данных (read_input), вывод строки CSV (write_row), диагностика сохранения энергии и импульса (diagnostics), собственный барьер (barrier_wait) и потоковая функция (thread_work). Запуск и управление потоками выполняет main.

4.2. Ввод и вывод данных

Ввод (функция read_input): выделяется память под массивы масс, координат, скоростей и сил, затем из файла читаются n частиц по 7 чисел каждая.

```
m     = malloc(n      * sizeof(double));
pos   = malloc(3 * n  * sizeof(double));
vel   = malloc(3 * n  * sizeof(double));
force = malloc(3 * n  * sizeof(double));

for (int q = 0; q < n; q++) {
    if (fscanf(f, "%lf %lf %lf %lf %lf %lf %lf",
```

```
                &m[q], &pos[3*q], &pos[3*q+1], &pos[3*q+2],
                &vel[3*q], &vel[3*q+1], &vel[3*q+2]) != 7) {
            fclose(f);
            return 0;
        }
    }
```

Координаты хранятся «упакованно»: для частицы q три координаты лежат в pos[3*q], pos[3*q+1], pos[3*q+2] (x, y, z). Так же устроены vel и force.

Вывод (функция write_row): в файл записывается время и координаты всех частиц через пробел - это одна строка CSV.

4.3. Параллельная схема и этап сил

Работа между потоками делится двумя способами. Для этапа сил пары (q, k) распределяются так, чтобы каждый поток получил примерно равное число пар (балансировка нагрузки, функции cum_pairs и find_q). Для этапа сведения и обновления частицы делятся на непересекающиеся блоки.

Ключевой фрагмент - вычисление сил с использованием третьего закона Ньютона (функция thread_work):

```
for (q = frc_start[t]; q < frc_end[t]; q++) {
    double mq   = m[q];
    double px   = pos[3*q];
    double py   = pos[3*q+1];
    double pz   = pos[3*q+2];
    for (k = q + 1; k < n; k++) {
        double dx = pos[3*k]   - px;
        double dy = pos[3*k+1] - py;
        double dz = pos[3*k+2] - pz;
        double d2 = dx*dx + dy*dy + dz*dz;
        double inv3 = 1.0 / (d2 * sqrt(d2));   /* 1/r^3 */
        double f = G * mq * m[k] * inv3;
        double fx = f * dx, fy = f * dy, fz = f * dz;
        lf[3*q]   += fx;  lf[3*q+1]   += fy;  lf[3*q+2]   += fz;
        lf[3*k]   -= fx;  lf[3*k+1]   -= fy;  lf[3*k+2]   -= fz;
    }
}
```

Каждый поток накапливает силы в своём локальном массиве lforce[t]. Это исключает гонки данных: ни один поток не пишет в чужой массив.

Сведение и обновление выполняет каждый поток для своих частиц:

```
for (u = 0; u < thread_count; u++)
    F += lforce[u][3*q + dim];
force[3*q + dim] = F;
double old_v = vel[3*q + dim];
pos[3*q + dim]  += old_v * dt;            /* x^n = x^(n-1) + v^(n-1)*dt */
vel[3*q + dim]  = old_v + (F / mq) * dt;  /* v^n = v^(n-1) + (F/m)*dt */
```

4.4. Барьер

Поток не должен начинать следующий шаг, пока другие не закончили предыдущий. Для этого используется барьер. На macOS нет стандартного pthread_barrier_*, поэтому барьер реализован самостоятельно на мьютексе и условной переменной:

```
pthread_mutex_lock(&bmutex);
int gen = bgeneration;
bcount++;
if (bcount == bthreads) {
    bgeneration++;
    bcount = 0;
    pthread_cond_broadcast(&bcond);
} else {
    while (gen == bgeneration)
        pthread_cond_wait(&bcond, &bmutex);
}
pthread_mutex_unlock(&bmutex);
```

Счётчик «поколений» generation защищает от ситуации, когда быстрый поток добегает до следующего барьера раньше, чем медленный покинул предыдущий; без него возникает взаимная блокировка (дедлок).

4.5. Метод Эйлера в коде

```
double old_v = vel[3*q + dim];
pos[3*q + dim]  += old_v * dt;            /* x^n = x^(n-1) + v^(n-1)*dt */
vel[3*q + dim]  = old_v + (F / mq) * dt;  /* v^n = v^(n-1) + (F/m)*dt */
```

5. Форматы данных

	Формат
Вход	n масса x y z vx vy vz ...
Выход (CSV)	t x1 y1 z1 x2 y2 z2 ... xn yn zn (строка на каждый момент времени)

6. Проверка корректности

- Сохранение суммарного импульса. Для внутренних сил, подчиняющихся третьему закону Ньютона, суммарный импульс $P = \Sigma\, m \cdot v$ обязан сохраняться точно. В прогонах импульс не меняется в пределах машинной точности - это подтверждает правильное применение третьего закона.
- Сверка с эталоном. Результат сравнивался с независимым эталонным интегратором (тот же метод Эйлера на Python). Расхождение конечных положений порядка 10^{-10} - совпадение.
- Детерминизм. При фиксированном числе потоков два запуска дают байт-в-байт одинаковый результат - гонок данных нет.
- Энергия. Для метода Эйлера энергия сохраняется приближённо и слабо дрейфует; дрейф уменьшается с уменьшением dt (ожидаемое поведение метода).

7. Результаты производительности

7.1. Аппаратная архитектура

Параметр	**Значение**
Процессор	Apple M3 (8 ядер)
Оперативная память	16 ГБ
Операционная система	macOS 26.6.2 (arm64)
Компилятор	Apple clang 21.0.0, флаги -O2
Средство замера времени	gettimeofday (макрос GET_TIME)

7.2. Методика замера

Измерялось полное время параллельных вычислений (этап сил + сведение + обновление) на задаче из 2000 частиц и 200 шагов интегрирования. Для каждой конфигурации выполнялось 5 запусков; в таблице приведены среднее и минимальное время. Ускорение $S = T_1 / T_p$ рассчитано по средним значениям, эффективность $E = S / p$.

7.3. Результаты

Потоков	**Среднее, с**	**Минимум, с**	**Ускорение S**	**Эффективность E**
1	1.444	1.423	1.00×	1.00
2	0.687	0.652	2.10×	1.05
4	0.478	0.413	3.02×	0.76
8	0.332	0.313	4.35×	0.54

Ускорение растёт с числом потоков, но сублинейно, что характерно для задачи N-тел: с ростом числа потоков увеличиваются накладные расходы на барьеры и сведение локальных сил, а также конкуренция за общую память.

8. Выводы

Реализовано параллельное решение задачи N-тел методом Эйлера первого порядка на потоках pthreads. Использование третьего закона Ньютона позволило вдвое сократить число вычислений сил и обеспечило точное сохранение суммарного импульса. Корректность подтверждена сверкой с независимым интегратором. Достигнуто ускорение до 4.35× на 8 потоках (Apple M3);